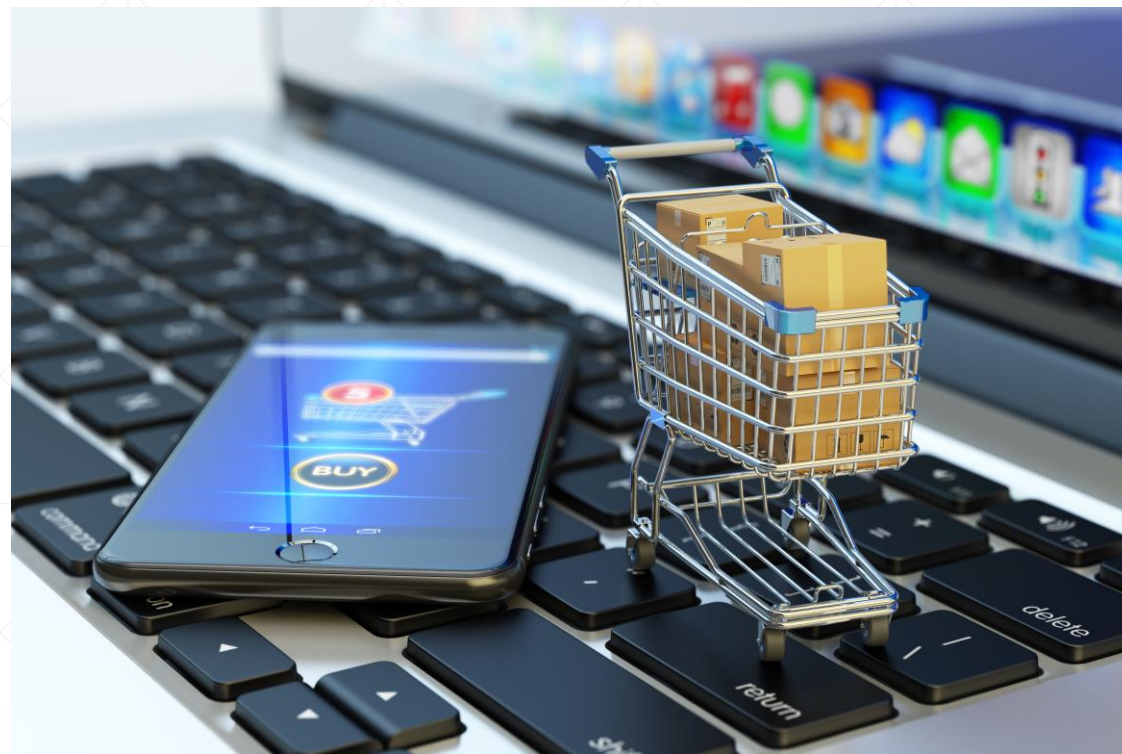




活用JDK中的函数式接口

北京理工大学计算机学院
金旭亮

Help Contents Search Related Topics Bookmarks Index

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP Java™ Platform Standard Ed. 8

PREV PACKAGE NEXT PACKAGE FRAMES NO FRAMES ALL CLASSES

Package java.util.function

Functional interfaces provide target types for lambda expressions and method references.

See: Description

Interface Summary

Interface	Description
BiConsumer<T,U>	Represents an operation that accepts two input arguments and returns no result.
BiFunction<T,U,R>	Represents a function that accepts two arguments and produces a result.
BinaryOperator<T>	Represents an operation upon two operands of the same type, producing a result of the same type as the operands.
BiPredicate<T,U>	Represents a predicate (boolean-valued function) of two arguments.
BooleanSupplier	Represents a supplier of boolean-valued results.
Consumer<T>	Represents an operation that accepts a single input argument and returns no result.
DoubleBinaryOperator	Represents an operation upon two double-valued operands and producing a double-valued result.
DoubleConsumer	Represents an operation that accepts a single double-valued argument and returns no result.
DoubleFunction<R>	Represents a function that accepts a double-valued argument and produces a result.
DoublePredicate	Represents a predicate (boolean-valued function) of one double-valued argument.
DoubleSupplier	Represents a supplier of double-valued results.
DoubleToIntFunction	Represents a function that accepts a double-valued argument and produces an int-valued result.
DoubleToLongFunction	Represents a function that accepts a double-valued argument and produces a long-valued result.

java.util.function包中定义了N多的函数式接口，可方便地与Lambda表达式特性相配合，写出可维护性强的代码。

这讲和后面几讲，将会选择几个在开发中经常要用到的接口进行介绍.....

学习指南-1



记住以下两点，有助于掌握JDK中诸多函数式接口的使用方法：

1

函数式接口，可用于定义变量，此变量可以引用Lambda表达式。

2

要执行“Lambda表达式”时，需要通过变量调用函数式接口所定义的唯一的公有抽象方法，并且传入相应的实参。

学习指南-2



JDK中所提供的诸多函数式接口，要想掌握好它们，必须注意以下三点：

1

了解接口的职责

2

弄清楚它接收几个参数，这些参数的类型是什么，如果有返回结果，这个结果的类型是什么

3

记住接口所定义的唯一抽象方法是哪个。

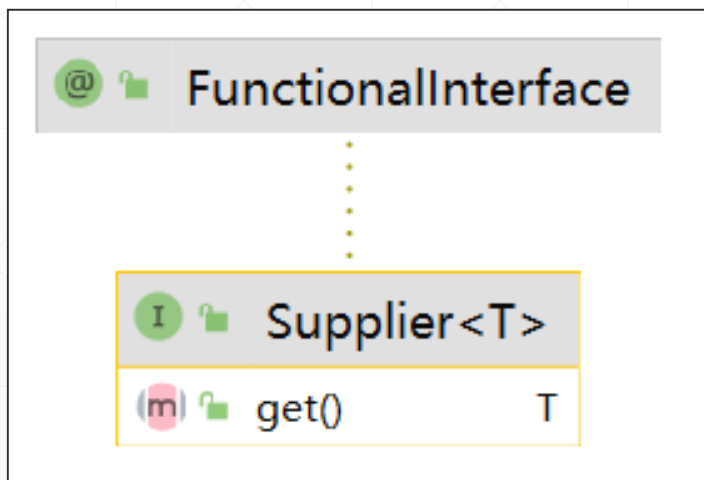


数据的生产与消费

使用Consumer和Supplier接口

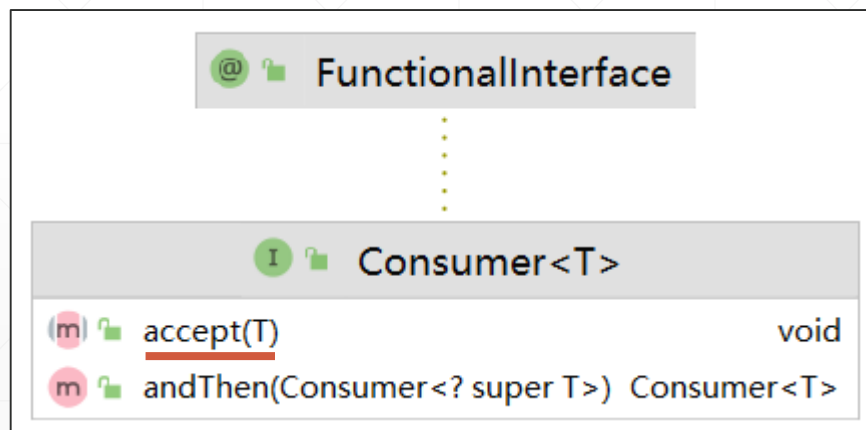
生产——<T>接口

当需要“生产”某种产品时，可以让类实现Supplier<T>接口，这个接口非常简单，只定义有一个get方法，返回一个指定类型T的对象：

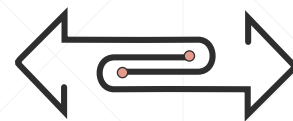


消费—Consumer<T>接口

当某对象需要“消费”特定类型的数据时，它需要实现Consumer<T>接口，此接口定义了一个accept()方法，它的参数引用那个“将被消费”的数据对象，此方法没有返回值：



生产与消费



```
private static void consumerAndSupplier() {  
    //消费者  
    Consumer<String> printer = System.out::println;  
    //生产者  
    Supplier<Integer> numFactory = () -> new Random().nextInt();  
    for (int i = 0; i < 5; i++) {  
        //生产者生产数据，消费者消费数据  
        printer.accept(numFactory.get().toString());  
    }  
}
```

生产者生产随机整数，消费者接收到这些数之后，把它输出到控制台窗口中。

Consumer接口中的默认方法

Consumer<T>接口中定义有一个默认方法，其源码如下：

```
default Consumer<T> andThen(Consumer<? super T> after) {  
    Objects.requireNonNull(after);  
    return (T t) -> { accept(t); after.accept(t); };  
}
```

从此方法的返回值可知，使用它可以将两个Consumer合成为一个复合的Consumer，外界执行这个Consumer，将会导致它所包容的两个子Consumer顺序执行.....

级联的Consumer<T>

Consumer接口中的andThen()方法可用于“级联”多个Consumer类型的Lambda表达式，按顺序执行它们：

```
private static void consumerAndThen() {  
    Consumer<String> consumer1 = info ->  
        System.out.println("First Consumer:" + info);  
    //级联两个Consumer  
    var consumer2 : Consumer<String> = consumer1.andThen(  
        info -> System.out.println("Second Consumer:"  
            + info.toUpperCase()));  
    consumer2.accept(t: "Hello");  
}
```



```
First Consumer:Hello  
Second Consumer:HELLO
```

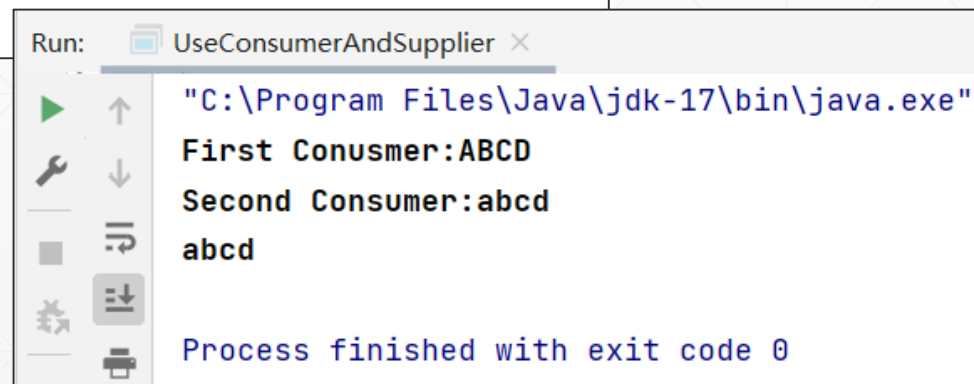
级联多个Consumer

使用andThen这种方式，可以很方便地将多个Consumer连接成一个长链，每个Consumer完成不同的处理工作，彼此互不影响.....

```
private static void consumerAndThen2() {  
    //定义三个独立的Consumer  
    Consumer<String> consumer1 = info ->  
        System.out.println("First Conusmer:" + info.toUpperCase());  
    Consumer<String> consumer2 = info -> System.out.println("Second Consumer:"  
        + info.toLowerCase());  
    Consumer<String> finalConsumer = System.out::println;  
    //级联三个Consumer，但都是“各人自扫门前雪”，互不影响  
    consumer1.andThen(consumer2).andThen(finalConsumer).accept(t: "abcd");  
}
```

实参是最后传入的

运行结果：

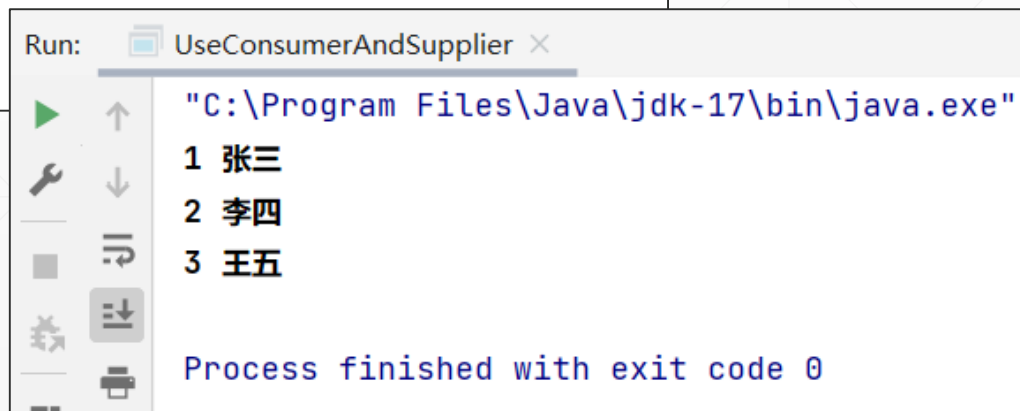


```
Run: UseConsumerAndSupplier x  
C:\Program Files\Java\jdk-17\bin\java.exe  
First Conusmer:ABCD  
Second Consumer:abcd  
abcd  
Process finished with exit code 0
```

andThen的一个应用实例

```
private static void printStudentInfo() {  
    //创建一个用于测试的学生对象集合  
    List<Student> students = new ArrayList<>();  
    students.add(new Student(id: 1, name: "张三"));  
    students.add(new Student(id: 2, name: "李四"));  
    students.add(new Student(id: 3, name: "王五"));  
    //创建两个Consumer, 分别输出学生的学号与姓名  
    Consumer<Student> printId = student -> System.out.print(student.getId() + " ");  
    Consumer<Student> printName = student -> System.out.println(student.getName());  
    //遍历学生集合, 依次输出其信息  
    students.forEach(printId.andThen(printName));  
}
```

运行结果:



```
Run: UseConsumerAndSupplier x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
1 张三  
2 李四  
3 王五  
Process finished with exit code 0
```

不要这么干!

依据Consumer的原意，它“只消费数据”而“不修改数据”，因此，不要写代码修改集合中的原始数据!



```
private static void consumeAndModify() {  
    List<Student> students = new ArrayList<>();  
    students.add(new Student( id: 1, name: "张三"));  
    students.add(new Student( id: 2, name: "李四"));  
    students.add(new Student( id: 3, name: "王五"));  
  
    Consumer<Student> printAndModifyId = student -> {  
        student.setId(student.getId() * 2);  
        System.out.print(student.getId() + " ");  
    };  
  
    Consumer<Student> printAndModifyName =  
        student -> {  
            student.setName(student.getName() + " "  
                + new Random().nextInt());  
            System.out.println(student.getName());  
        };  
  
    students.forEach(printAndModifyId.andThen(printAndModifyName));  
}
```

巩固练习



JDK中定义了另外一些Consumer和Supplier接口，列表如下，请自学这些接口，并且为它们设计一些实例，掌握其用法。

接口	单一抽象方法
IntConsumer	<code>void accept(int x)</code>
DoubleConsumer	<code>void accept(double x)</code>
LongConsumer	<code>void accept(long x)</code>
BiConsumer	<code>void accept(T t, U u)</code>

接口	单一抽象方法
IntSupplier	<code>int getAsInt()</code>
DoubleSupplier	<code>double getAsDouble()</code>
LongSupplier	<code>long getAsLong()</code>
BooleanSupplier	<code>boolean getAsBoolean()</code>

下一讲



Predicate接口与对象查找